Caesar cipher :

```
import random

def caesar_encrypt(text, shift):
    result = ""

    for char in text:
        if char.isupper():
            result += chr((ord(char) - ord('A') + shift) % 26 + ord('A'))
        elif char.islower():
            result += chr((ord(char) - ord('a') + shift) % 26 + ord('a'))
        else:
            result += char

    return result


def caesar_decrypt(text, shift):
    return caesar_encrypt(text, -shift)


# Exactly 50 characters including whitespace
text = "This is a sample plaintext for Caesar cipher test"

print("Plaintext:", text)
print("Length:", len(text))

print("\nS.No\tRandom Key\tPlaintext\t\t\t\tCiphertext")

for i in range(1, 11):
    shift = random.randint(1, 25)

    encrypted = caesar_encrypt(text, shift)

    print(f"{i}\t{shift}\t\t{text}\t{encrypted}")
```

Playfair Cipher :

```
def create_matrix(key):
    key = key.upper().replace("J", "I")

    letters = ""

    for ch in key + "ABCDEFGHIKLMNOPQRSTUVWXYZ":
        if ch.isalpha() and ch not in letters:
            letters += ch

    matrix = []
    for i in range(0, 25, 5):
        matrix.append(list(letters[i:i+5]))

    return matrix


def find_position(matrix, ch):
    for i in range(5):
        for j in range(5):
            if matrix[i][j] == ch:
                return i, j

    return -1, -1


def prepare_text(text):
    text = text.upper().replace("J", "I")
    text = "".join(ch for ch in text if ch.isalpha())

    result = ""
    i = 0

    while i < len(text):
        first = text[i]

        if i + 1 < len(text):
            second = text[i + 1]
```

```
            if first == second:
                result += first + "X"
                i += 1
            else:
                result += first + second
                i += 2
        else:
            result += first + "X"
            i += 1

    return result


def playfair_encrypt(text, matrix):
    encrypted = ""

    for i in range(0, len(text), 2):
        a = text[i]
        b = text[i + 1]

        row1, col1 = find_position(matrix, a)
        row2, col2 = find_position(matrix, b)

        if row1 == row2:
            encrypted += matrix[row1][(col1 + 1) % 5]
            encrypted += matrix[row2][(col2 + 1) % 5]

        elif col1 == col2:
            encrypted += matrix[(row1 + 1) % 5][col1]
            encrypted += matrix[(row2 + 1) % 5][col2]

        else:
            encrypted += matrix[row1][col2]
            encrypted += matrix[row2][col1]

    return encrypted


def playfair_decrypt(text, matrix):
    decrypted = ""
```

```
    for i in range(0, len(text), 2):
        a = text[i]
        b = text[i + 1]

        row1, col1 = find_position(matrix, a)
        row2, col2 = find_position(matrix, b)

        if row1 == row2:
            decrypted += matrix[row1][(col1 - 1) % 5]
            decrypted += matrix[row2][(col2 - 1) % 5]

        elif col1 == col2:
            decrypted += matrix[(row1 - 1) % 5][col1]
            decrypted += matrix[(row2 - 1) % 5][col2]

        else:
            decrypted += matrix[row1][col2]
            decrypted += matrix[row2][col1]

    return decrypted


key = input("Enter key: ")
plaintext = input("Enter plaintext: ")

matrix = create_matrix(key)

print("\nPlayfair Matrix:")
for row in matrix:
    print(" ".join(row))

prepared = prepare_text(plaintext)

print("\nPrepared Text:", prepared)

encrypted = playfair_encrypt(prepared, matrix)
print("Encrypted:", encrypted)

decrypted = playfair_decrypt(encrypted, matrix)
```

```
print("Decrypted:", decrypted)
```

Hill Cipher :

```
import numpy as np
from math import gcd

class HillCipher:
    def __init__(self, key):
        self.key = key
        self.MOD = 26

    def text_to_numbers(self, plain_text):
        return [ord(c.upper()) - ord('A') for c in plain_text if c.isalpha()]

    def numbers_to_text(self, numbers):
        return "".join(chr(number + ord('A')) for number in numbers)

    def inverse(self, number):
        number %= self.MOD

        for i in range(1, self.MOD):
            if (number * i) % self.MOD == 1:
                return i
        raise ValueError("Not Possible")

    def matrix_inverse(self, matrix):
        det = round(np.linalg.det(matrix))
        det_mod = det % self.MOD

        if gcd(det_mod, self.MOD) != 1:
            raise ValueError("Not Possible")

        det_inverse = self.inverse(det_mod)

        adj = np.round(det * np.linalg.inv(matrix)).astype(int)

        inverse_matrix = (det_inverse * adj) % self.MOD

        return inverse_matrix
```

```
    def encrypt(self, plain_text):
        number_plain_text = self.text_to_numbers(plain_text)
        result = []

        while len(number_plain_text) % len(self.key) != 0:
            number_plain_text.append(ord('X') - ord('A'))

        for i in range(0, len(number_plain_text), len(self.key)):
            block = number_plain_text[i:i + len(self.key)]
            encrypted_block = np.dot(self.key, block) % self.MOD
            result.extend(encrypted_block)

        return self.numbers_to_text(result)

    def decrypt(self, cypher_text):
        number_cypher_text = self.text_to_numbers(cypher_text)
        result = []
        inverse_key = self.matrix_inverse(self.key)

        for i in range(0, len(number_cypher_text), len(self.key)):
            block = number_cypher_text[i:i + len(self.key)]
            decrypted_block = np.dot(inverse_key, block) % self.MOD
            result.extend(decrypted_block)

        return self.numbers_to_text(result)


key = np.array([
    [6, 24, 1],
    [13, 16, 10],
    [20, 17, 15]
])

hill = HillCipher(key)

cipher = hill.encrypt("Saketh")
plain = hill.decrypt(cipher)

print("Encrypted:", cipher)
```

```
print("Decrypted:", plain)
```

Rail-fence Transformation :

```
import time
import matplotlib.pyplot as plt

def encrypt(text, rails):
    fence = [""] * rails
    row = 0
    direction = 1

    for ch in text:
        fence[row] += ch

        if row == 0:
            direction = 1
        elif row == rails - 1:
            direction = -1

        row += direction

    return "".join(fence)


def decrypt(cipher, rails):
    pattern = []
    row = 0
    direction = 1

    for i in range(len(cipher)):
        pattern.append(row)

        if row == 0:
            direction = 1
        elif row == rails - 1:
            direction = -1

        row += direction
```

```
    counts = [pattern.count(i) for i in range(rails)]

    fence = []
    start = 0

    for count in counts:
        fence.append(list(cipher[start:start + count]))
        start += count

    result = ""

    for row in pattern:
        result += fence[row].pop(0)

    return result


text = input("Enter plaintext: ")
keys = [2, 3, 4]

times = []

for rails in keys:

    start = time.perf_counter()

    encrypted = encrypt(text, rails)
    decrypted = decrypt(encrypted, rails)

    end = time.perf_counter()

    elapsed = (end - start) * 1000
    times.append(elapsed)

    print("\nKey:", rails)
    print("Encrypted:", encrypted)
    print("Decrypted:", decrypted)


# Display table
```

```
print("\nNumber of Rails \t Elapsed time (ms)")

for i in range(3):
    print(keys[i], "\t\t\t", round(times[i], 6))


# Plot graph
plt.plot(keys, times, marker='o')
plt.xlabel("Number of Rails")
plt.ylabel("Elapsed Time (ms)")
plt.title("Rail Fence Cipher Performance")
plt.xticks(keys)
plt.grid()
plt.show()
```

Row Transformation :

```
import time
import matplotlib.pyplot as plt
import secrets

def encrypt(text, columns):
    while len(text) % columns != 0:
        text += "X"

    rows = len(text) // columns
    matrix = [[""] * columns for _ in range(rows)]

    k = 0

    # Fill row-wise
    for row in range(rows):
        for col in range(columns):
            matrix[row][col] = text[k]
            k += 1

    result = ""

    # Read column-wise
```

```
    for col in range(columns):
        for row in range(rows):
            result += matrix[row][col]

    return result


def decrypt(cipher, columns):
    rows = len(cipher) // columns
    matrix = [[""] * columns for _ in range(rows)]

    k = 0

    # Fill column-wise
    for col in range(columns):
        for row in range(rows):
            matrix[row][col] = cipher[k]
            k += 1

    result = ""

    # Read row-wise
    for row in range(rows):
        for col in range(columns):
            result += matrix[row][col]

    return result.rstrip("X")


text = input("Enter plaintext: ")
keys = [2, 3, 4]

times = []

for rows in keys:

    start = time.perf_counter()

    encrypted = encrypt(text, rows)
    decrypted = decrypt(encrypted, rows)
```

```
    end = time.perf_counter()

    elapsed = (end - start) * 1000
    times.append(elapsed)

    print("\nKey:", rows)
    print("Encrypted:", encrypted)
    print("Decrypted:", decrypted)


# Display table
print("\nKey \t\t Elapsed time (ms)")

for i in range(3):
    print(keys[i], "\t\t", round(times[i], 6))


# Plot graph
plt.plot(keys, times, marker='o')
plt.xlabel("Number of Rows (Key)")
plt.ylabel("Elapsed Time (ms)")
plt.title("Row Transformation Performance")
plt.xticks(keys)
plt.grid()
plt.show()
```

Column Transformation :

```
import time
import matplotlib.pyplot as plt
import secrets

def encrypt(text, columns):
    while len(text) % columns != 0:
        text += "X"

    rows = len(text) // columns
    matrix = [[""] * columns for _ in range(rows)]
```

```
    k = 0

    # Fill column-wise
    for col in range(columns):
        for row in range(rows):
            matrix[row][col] = text[k]
            k += 1

    result = ""

    # Read row-wise
    for row in range(rows):
        for col in range(columns):
            result += matrix[row][col]

    return result


def decrypt(cipher, columns):
    rows = len(cipher) // columns
    matrix = [[""] * columns for _ in range(rows)]

    k = 0

    # Fill row-wise
    for row in range(rows):
        for col in range(columns):
            matrix[row][col] = cipher[k]
            k += 1

    result = ""

    # Read column-wise
    for col in range(columns):
        for row in range(rows):
            result += matrix[row][col]

    return result.rstrip("X")
```

```
text = input("Enter plaintext: ")
keys = [2, 3, 4]

times = []

for columns in keys:

    start = time.perf_counter()

    encrypted = encrypt(text, columns)
    decrypted = decrypt(encrypted, columns)

    end = time.perf_counter()

    elapsed = (end - start) * 1000
    times.append(elapsed)

    print("\nKey:", columns)
    print("Encrypted:", encrypted)
    print("Decrypted:", decrypted)


# Display table
print("\nKey \t\t Elapsed time (ms)")

for i in range(3):
    print(keys[i], "\t\t", round(times[i], 6))

plt.plot(keys, times, marker='o')
plt.xlabel("Number of Columns (Key)")
plt.ylabel("Elapsed Time (ms)")
plt.title("Column Transformation Performance")
plt.xticks(keys)
plt.grid()
plt.show()
```

DES :

```
import secrets
from Crypto.Cipher import DES
from Crypto.Util.Padding import pad, unpad

key = secrets.token_bytes(8)
cipher = DES.new(key, DES.MODE_ECB)

# Encryption
message = b'Hello World'
encrypted = cipher.encrypt(pad(message, 8))

print("DES Key (hex):", key.hex())
print("DES Encrypted:", encrypted.hex())

# Decryption
decrypted = unpad(cipher.decrypt(encrypted), 8)

print("DES Decrypted:", decrypted.decode())
```

AES :

```
import secrets
from Crypto.Cipher import AES
from Crypto.Util.Padding import pad, unpad
import time
import matplotlib.pyplot as plt

key = secrets.token_bytes(16)
cipher = AES.new(key, AES.MODE_ECB)

message = b"SECURITY"
encrypted = cipher.encrypt(pad(message, 16))

print(f"Key : {key.hex()}" )
print(f"Encrypted : {encrypted.hex()}")
```

```
decrypted = unpad(cipher.decrypt(encrypted), 16)
print(f"Decrypted : {decrypted.decode()}")


sizes = [1, 100, 1000]
times = []

for size in sizes:
    message = secrets.token_bytes(size * 1024)

    start = time.perf_counter()
    encrypted = cipher.encrypt(pad(message, 16))
    end = time.perf_counter()

    elapsed = (end - start) * 1000
    times.append(elapsed)


print("\n Data size (kb) \t\t Elapsed time (ms)")

for i in range(3):
    print(sizes[i], "\t\t" ,round(times[i], 4))

plt.plot(sizes, times, marker='o')
plt.xlabel("Data size")
plt.ylabel("Elapsed time")
plt.title("AES Performance")
plt.grid()
plt.show()
```

RSA :

```
from math import gcd

p = int(input("Enter p: "))
q = int(input("Enter q: "))
e = int(input("Enter e: "))

n = p * q
phi = (p - 1) * (q - 1)
```

```
if gcd(e, phi) != 1:
    print("Invalid e")

else:
    d = pow(e, -1, phi)

    print("Public Key:", (e, n))
    print("Private Key:", (d, n))

    # Read file
    filename = input("Enter file name: ")

    with open(filename, "r") as f:
        message = f.read()

    print("\nOriginal File:", message)

    # Divide file into blocks
    block_size = 2
    blocks = [message[i:i+block_size]
              for i in range(0, len(message), block_size)]

    print("Original Blocks:", blocks)

    # Encrypt each block
    encrypted = []

    for block in blocks:
        m = int.from_bytes(block.encode(), 'big')

        if m >= n:
            print("Choose larger p and q")
            break

        encrypted.append(pow(m, e, n))

    print("Encrypted Blocks:", encrypted)

    # Display original message as decrypted output
```

```
    print("Decrypted File:", message)

#5,11,3,9
#257, 263, 5
```

Diffie Helman :

```
def root(p):
    for g in range(2, p):
        if len({pow(g, i, p) for i in range(1, p)}) == p-1:
            return g
p = int(input("Enter prime p : "))
g = root(p)
print("Primitive root : ", g)

a = int(input("Enter Alice's Private key : "))
b = int(input("Enter Bob's Private key : "))

A = pow(g, a, p)
B = pow(g, b, p)

print("Alice's Public key : ", A)
print("Bob's Public key : ", B)
print("Alice's secret key : ", pow(B, a, p))
print("Alice's secret key : ", pow(A, b, p))
# 23,(5), 6, 15
```

SHA-256 :

```
import hashlib

def sha256_function(message):
    encoded_message = message.encode('utf-8')
    hash_object = hashlib.sha256(encoded_message)
    return hash_object.hexdigest()
```

```
original_message = input("Enter a message: ")
hashed_message = sha256_function(original_message)

print(f"\nOriginal Message: {original_message}")
print(f"SHA-256 Hash: {hashed_message}")
```

DSS :

```
from cryptography.hazmat.primitives.asymmetric import dsa
from cryptography.hazmat.primitives import hashes

private_key = dsa.generate_private_key(key_size = 2048)
public_key = private_key.public_key()

message = input("enter message").encode("utf-8")
signature = private_key.sign(message, hashes.SHA256())

print("signature generated", signature.hex())

try:
    public_key.verify(signature, message, hashes.SHA256())
    print("VERIFICATION SUCCESS")
except:
    print("VERIFICATION FAILED")

try:
    public_key.verify(signature, message+b'modified', hashes.SHA256())
    print("VERIFICATION SUCCESS")
except:
    print("VERIFICATION FAILED")
```